When Meta-Learned Optimization Fails to Beat Round-Robin: A Validation Study on Hyperbolic Polytope Volume

Igor Rivin

August 15, 2026

Superseded validation result (2026-08-15). The evaluation-accounting audit found that the baseline runner charged padded history lengths rather than actual objective, surrogate, and gradient work. The "matched budget" claims and numerical comparisons below are therefore not valid current evidence and must not be cited or published. The source is retained for provenance; rerun the study with the repaired validator and retrained controllers before replacing this notice.

Abstract

We investigate whether a reinforcement-learning meta-controller can outperform fixed single-optimizer and uniform-mixing baselines on a family of optimization tasks parameterized by dimension d and vertex count N (maximizing the hyperbolic volume of an ideal polytope in $\mathbb{H}^d$). A Gymnasium environment with a hierarchical search tree dispatches calls to a small library of inner optimizers (CMA-ES on the Euclidean surrogate, L-BFGS on closed-form hyperbolic gradients, plus generic sample / split / perturb / prune operations); a PPO policy is trained per dimension for 200,000 environment steps. Evaluated at matched budget on 200 tasks across $d \in \{3, 4, 5, 6\}$, the trained controller wins cleanly only at $d = 5$ (+15.9% geometric-mean ratio over the best single baseline). At $d = 3$ and $d = 4$ it loses to a round-robin baseline that splits the budget equally across the same inner optimizers; at $d = 6$ it is dominated by a single specialized solver (L-BFGS on the Murakami $\mathbb{H}^6$ volume), with catastrophic failure at the simplex-boundary case $N = 7$. We report the negative result honestly: meta-optimization adds value where no single classical heuristic dominates, and round-robin suffices everywhere else. All code, saved policies, and validation harness are released so the reward and observation design can be iterated on by other researchers.

1 Setup

The volume-maximization problem family parameterized by (d, N) is a natural target for meta-learned optimization: each task asks the same question (find a high-volume ideal polytope) with a shared structure (points on a sphere, a few closed-form objective and gradient routines, a handful of useful inner optimizers), and optimizer choice is clearly non-uniform across tasks—CMA-ES on the Euclidean surrogate dominates at small N, L-BFGS on the hyperbolic objective dominates once the configuration is close to optimal, and local sampling plus region splitting help escape basins at larger N. We implemented a hierarchical reinforcement-learning harness to test whether a learned meta-controller can outperform fixed single-optimizer and uniform-mixing baselines at matched evaluation budget.

Harness. The harness is a Gymnasium environment wrapping a hierarchical search tree. Each state holds the frontier of active regions of the configuration space $(S^{d-1})^N$; the agent selects a region and an operation (sample, split, perturb, prune, or invoke one of the inner optimizers). The reward is the improvement in the best-so-far volume minus a small cost per function evaluation. A PPO policy (stable-baselines3, MLP backbone) is trained per dimension for 200,000 environment steps, which corresponds to roughly 40–400 hours of simulated optimization depending on the inner-optimizer cost. The same harness supports problems orthogonal to polytope volume (we use it for cubic-graph girth maximization), which is the sense in which it is problem-agnostic.

Inner optimizers. For the polytope task family the inner-optimizer slate is:

- **CMA-ES on Euclidean surrogate:** fast, low-cost per evaluation; targets the convex-hull volume of the inscribed polytope. Effective at small N where the optimum is algebraically trivial.
- **L-BFGS on the hyperbolic objective:** closed-form gradients via Bloch–Wigner ($d = 3$), the Haagerup–Munkholm formula ($d = 4$), the Schläfli surrogate ($d = 5$), or the Murakami $\mathbb{H}^6$ volume ($d = 6$). Slow per call but converges to high-precision local maxima.
- **Sample, split, perturb, prune:** generic tree-management actions.

Fix to a latent scoring bug. The pre-refactor environment tracked a mix of Euclidean and hyperbolic volumes in the same node-best field at $d \in \{3, 4, 6\}$, because the CMA path scored Euclidean while the L-BFGS path returned hyperbolic. In the refactored harness used for the present study, each task has one consistent dimension-specific scoring function (closed-form hyperbolic for $d \in \{3, 4, 6\}$; Euclidean at $d = 5$, where it agrees with the hyperbolic optimum to full precision). All baseline numbers below are reported under the corrected scoring.

2 Validation Grid and Baselines

We test on a $4 \times 5 \times 10$ grid: $d \in \{3, 4, 5, 6\}$, five N values per d chosen to span small N where the global optimum is algebraically trivial (the regular simplex at $N = d+1$, the 4-simplex bipyramid, etc.) and larger N where combinatorial variety is rich, and 10 independent seeds per (d, N). Three methods are compared at matched budget (the evaluation count consumed by one RL episode):

(i) **RL:** the trained PPO controller;

(ii) **Round-robin:** the same total budget split equally across the inner optimizers available to the RL agent, ignoring context;

(iii) **Solo:** each single inner optimizer given the whole budget; we report the best single solo per task.

The pre-registered decision rule in our protocol is: RL passes the *learning-allocation* test if and only if it beats round-robin, because round-robin already exposes the same diversity of inner optimizers without any learning.

Table 1: Geometric-mean ratio to the best single solo baseline, per dimension, over 50 tasks (5 N's $\times$ 10 seeds). Values > 1 mean the method beats the strongest fixed optimizer.

d	RL	Round-robin	Dominant single-solo baseline
3	1.057	**1.081**	L-BFGS on Bloch–Wigner
4	1.132	**1.238**	CMA-ES on Euclidean surrogate
5	**1.159**	1.049	CMA-ES on Euclidean surrogate
6	0.580	0.529	**L-BFGS on Murakami** $\mathbb{H}^6$ (ratio 0.980)

3 Results

The outcome splits sharply by dimension (Table 1).

The RL controller wins cleanly only at $d = 5$. At $d = 3$ and $d = 4$ the uninformed round-robin baseline—splitting the budget equally across inner optimizers—beats the trained PPO policy by 2–9%. At $d = 6$ a single specialized inner optimizer (L-BFGS on the Murakami hyperbolic volume) is dominant: both RL and round-robin are far below it.

The simplex-boundary failure at $d = 6, N = 7$. The $d = 6$ deficit is driven primarily by the smallest N in the grid. At $N = 7$ (the regular ideal 6-simplex), the solo L-BFGS baseline finds the global optimum $V \approx 0.0067$ in a few gradient steps, while the RL median is 0.0005—a factor of 13 below. For $N \in \{10, 16, 20, 24\}$ at $d = 6$ the RL controller ties the solo-L-BFGS baseline to three significant digits, indicating that the learned policy *can* identify the right specialized solver in the large-N regime but does not recognize the simplex-boundary special case at small N. The pattern reproduces, less dramatically, at $d \in \{3, 4, 5\}$:

d	$N_{\min}$ (simplex)	RL/solo at $N_{\min}$	RL/solo for largest N
3	5	1.000	1.081
4	5	0.977	1.192
5	6	0.905	1.287
6	7	**0.079**	1.001

The penalty for RL over-exploration grows sharply with d at the simplex boundary, suggesting the policy has not learned to "defer" to the dominant inner optimizer when the configuration is close to a known closed-form optimum.

4 Narrow Claim and Failure Modes

The usable takeaway is more modest than "RL beats classical optimization on hyperbolic polytope volume":

1. **Where RL adds value:** at $d = 5$, the Euclidean–hyperbolic volume equivalence is exact (no inner optimizer dominates uniformly across N), and the learned controller captures enough of the right heuristic (+10.5% over round-robin, +15.9% over best solo baseline) to outperform uninformed mixing.

2. **Where RL ties or loses:** at $d \in \{3, 4\}$ a strong solo baseline (L-BFGS on Bloch–Wigner; CMA-ES on Euclidean) suffices, and round-robin already exposes the diversity that RL would need to learn. The trained policy contributes nothing beyond its noise floor.

3. **Where RL fails badly:** at $d = 6$ the closed-form Murakami gradient is dominant; the simplex-boundary case $N = 7$ exposes a specific failure mode in the learned policy (over-exploration when convergence is needed).

This is consistent with the broader meta-learning literature: meta-optimization pays off where classical heuristics are weak; where a strong specialized solver exists, the learned policy has to discover that solver's dominance and defer to it, which the current policy does only partially.

Why round-robin is so strong. The strength of round-robin in our setting is itself an informative finding. The inner-optimizer slate is small (4 to 6 options), each option is itself reasonable, and the per-call cost varies by $\sim 10\times$ across optimizers. Allocating the budget uniformly across them implicitly hedges across regimes: when CMA-ES dominates (small N, easy landscape) it gets a quarter of the budget and that suffices; when L-BFGS dominates (close to optimum) it likewise gets enough. A learned policy that is no better than uniform allocation is failing to add the context-dependent allocation that justifies its training cost.

5 Reproducibility

The full training scripts, saved policies, and validation harness are released at the repository accompanying this note. The training command is

```
python scripts/train_polytope_controller.py --d <D> --steps 200000
```

producing `ppo_d<D>_v2_final.zip` plus a configuration sidecar. The validation grid is reproduced by

```
python scripts/validate_rl_controller.py \
    --model-template 'results/rl_models/ppo_d{d}_v2_final.zip' \
    --dims 3,4,5,6 --seeds 10
```

which writes a JSON of every (method, d, N, seed) result, plus the aggregate ratios reported above. Both scripts share the same harness, so a researcher who wants to test a different reward function, observation encoding, or inner-optimizer slate can change them in one place and rerun the full grid in roughly a day of CPU time on a modest workstation.

6 Future Work

Three directions seem worth pursuing:

1. **Reward shaping.** The current reward is the increment in best-so-far volume. A potential-based shaping that includes a constant per-step penalty for over-exploration at the simplex boundary, or a curriculum that starts at large N where RL has its advantage, might rescue the small-N cases.

2. **Observation features.** The current observation is a flat vector of per-region statistics (depth, mean, variance, best, etc.) plus task metadata $(d, N, \log Nd)$. Adding a symmetry-invariant fingerprint of the current best configuration—e.g., the multiset of pairwise chordal distances—would let the policy recognize "this is close to a regular simplex" and defer to the corresponding solo optimizer.

3. **Comparison against off-the-shelf hyperparameter tuners.** Optuna and similar tools target a related but not identical task (single-study tuning, not learned meta-control over a task family). Benchmarking against them on the same (d, N) grid would establish whether our negative result is specific to our PPO setup or generalizes across meta-learning approaches.